

Complete Project Repository Setup

To share your API with the world, follow these final steps to prepare your code directory, document your project, and publish your work to a public GitHub repository.

1. Create a `.gitignore` File

Before pushing your code, you must prevent temporary dependencies from being uploaded to GitHub. Create a file named `.gitignore` in your project root directory and add the following contents:

Node.js Lane

text

```
node_modules/  
.env
```

Use code with caution.

Python Lane

text

```
__pycache__/  
.pytest_cache/  
.env  
venv/  
env/
```

Use code with caution.

2. Create the `README.md` File

Create a file named `README.md` in your project root directory. Copy and paste the template below, replacing the placeholder information with your actual text and values.

markdown

Task Management API

A lightweight, high-performance CRUD REST API built as a professional foundation for task

🚀 How to Install & Run

Get the API running locally on your machine in under two minutes.

Prerequisites

Make sure you have **Node.js** (v18+) or **Python** (v3.9+) installed on your system.

One-Command Start

Run the following single command in your terminal to install dependencies and boot up the

<!-- Choose the code block matching your chosen lane -->

```
```bash
```

```
For Node.js (Express) Lane:
```

```
npm install && node server.js
```

```
For Python (FastAPI) Lane:
```

```
pip install fastapi uvicorn pydantic && uvicorn main:app --reload --port 8000
```

```
```
```

Once running, the interactive documentation is available at:

* **Swagger UI Docs:** <http://localhost:3000/docs> (or port 8000 for FastAPI)

📖 API Endpoint Reference

| Method | Endpoint | Description | Success Code | Error Codes |
|---------------|-------------------------|---|-----------------------------|---------------------------|
| ---- | ---- | ---- | ---- | ---- |
| GET | <code>/</code> | Returns API name, version, and metadata | <code>200 OK</code> | None |
| GET | <code>/health</code> | Server availability monitor check | <code>200 OK</code> | None |
| GET | <code>/tasks</code> | Retrieves the entire collection of tasks | <code>200 OK</code> | None |
| GET | <code>/tasks/:id</code> | Fetches a single task by its unique ID | <code>200 OK</code> | <code>404 Not Fo</code> |
| POST | <code>/tasks</code> | Creates a new task with validation | <code>201 Created</code> | <code>400 Bad Requ</code> |
| PUT | <code>/tasks/:id</code> | Replaces/updates title or completed state | <code>200 OK</code> | <code>400 Bad</code> |
| DELETE | <code>/tasks/:id</code> | Permanently deletes a task by ID | <code>204 No Content</code> | <code>404 Not</code> |

🖥️ Sample Terminal Output

Here is a live receipt showing validation enforcement when retrieving a specific existing

```
```bash
\$ curl -i http://localhost:3000/tasks/1

HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 61
ETag: W/"3d-B8o/b7r2yB9z6w1U4lDk"
Date: Sun, 23 Aug 2026 18:55:00 GMT
Connection: keep-alive

{"id":1,"title":"Buy groceries","done":false}
```

---
```

📄 Interactive Documentation

Below is a snapshot of our fully interactive Swagger UI console showing live endpoint mapp

![Swagger UI Screen Capture](https://unsplash.com)
*(Note: Replace this placeholder placeholder URL with your real uploaded repository screen

Use code with caution.

3. Push Progress to GitHub

Run these final terminal commands to lock in your work, create your final commit stage, and push your branch out to GitHub:

```
bash

# 1. Stage the final changes (README and .gitignore)
git add .
```

2. Record your Stage 6 milestone commit

```
git commit -m "Stage 6: publish and docs"
```

3. Create a brand new Public repository on github.com (do not initialize with README)

4. Link your Local project directory to your new remote GitHub repo

```
git remote add origin https://github.com
```

5. Rename your primary default Local development branch to main

```
git branch -M main
```

6. Push all 6 chronological development stages to the cloud!

```
git push -u origin main
```